MATH DIRECTED READING PROGRAM / MEETING 4

From Weighted Averages to Attention

Self-contained guided reading and problem sheet | Revision 3.0

Aim. Start with a weighted average, let its coefficients depend on an input, and then learn the maps that construct those coefficients. Only after the objects are defined do we name them queries, keys, and values.

Background and pacing. Inner products, differentiation of quadratics, exponentials, and softmax. The probability needed for the scaling argument is reviewed in 4.4. Problems 4.1–4.3 introduce the operation; 4.5 connects it to training. The variance review in 4.4 and the full optional derivative can be discussed separately. Individual vectors are columns in this sheet.

Using the self-checks. After each numbered problem, try its short self-checks with the worked solution covered. Definitions may stay open. Give a reason, not only a yes/no answer; these checks guide discussion rather than add a timed test.

4.1 Why does a weighted average appear?

REVIEW: NONNEGATIVE COEFFICIENTS AND A QUADRATIC

Given scalar observations $v_1, \dots, v_n$, suppose we want one number u to summarize them. Choose coefficients $a_j \geq 0$ with $A = \sum_j a_j > 0$, and measure disagreement by the **weighted least-squares objective**

$$J(u) = \frac{1}{2} \sum_{j=1}^{n} a_j (u - v_j)^2.$$

Here the a_j are fixed while minimizing over u. Dividing them by A gives coefficients $\alpha_j = a_j/A$ that are nonnegative and sum to one. A sum $\sum_j \alpha_j v_j$ with such coefficients is a **convex combination**. Combining several inputs into one summary is often called **pooling** in machine learning.

Worked example. For $v_1 = 2$, $v_2 = 6$, $a_1 = 1$, and $a_2 = 3$, we have $J'(u) = (u - 2) + 3(u - 6) = 4u - 20$. Thus its unique minimizer is 5, closer to the observation with larger coefficient.

Guided questions

(a) Compute $J'(u)$ and $J''(u)$ for the general problem. Derive the unique minimizer

$$u^* = \frac{\sum_j a_j v_j}{\sum_j a_j} = \sum_j \alpha_j v_j.$$

(b) Why does multiplying all a_j by the same positive number leave u^* unchanged? What would go wrong with this formula if all a_j were zero?

(c) Prove $\min_j v_j \leq u^* \leq \max_j v_j$.

(d) Now let $v_j, u \in \mathbb{R}^r$ and replace $(u - v_j)^2$ by $\|u - v_j\|_2^2 = \sum_k (u_k - v_{j,k})^2$. Minimize coordinate by coordinate. Do the same coefficients average every coordinate?

Translate back into machine-learning language. A pooling operation can be understood as a minimizer of a simple quadratic. Nothing has been learned yet: in this problem the averaging coefficients were supplied in advance.

SELF-CHECK 4.1

1. A normalized weighted average combines scalar values 2 and 6. Can it equal 9? What mathematical property, not an ML slogan, rules this out?
2. For fixed coefficients a_1, a_2, is the summary u being optimized the same kind of object as a learned attention-projection parameter? Explain what was fixed in this minimization.

4.2 Let the coefficients depend on a location

SETUP: A LOCAL AVERAGE FOR A NEW INPUT

Suppose observations are pairs (k_j, v_j), with locations $k_j \in \mathbb{R}^d$ and associated scalar or vector outputs v_j. To predict at a new location $q \in \mathbb{R}^d$, give nearby locations greater coefficients. One choice is the **Gaussian kernel**

$$K_\tau(q,k) = \exp\left(-\frac{\|q-k\|_2^2}{2\tau^2}\right), \qquad \tau > 0.$$

The positive number τ is a **bandwidth**: it controls how quickly the coefficient decreases with distance. A kernel here means a chosen similarity-weighting function; no theory of reproducing-kernel Hilbert spaces is needed. Normalize and average:

$$\alpha_j(q) = \frac{K_\tau(q,k_j)}{\sum_l K_\tau(q,k_l)}, \qquad f(q) = \sum_j \alpha_j(q) v_j.$$

This construction is the **Nadaraya–Watson kernel regression estimator**. **Regression** means predicting a numerical output rather than a class. The weights now depend on the location where the prediction is requested.

Worked symmetry. In one dimension, take $k_1 = -1$, $k_2 = 1$, $v_1 = 0$, $v_2 = 10$. At $q = 0$ the distances agree, so $f(0) = 5$ for every bandwidth $\tau > 0$.

Guided questions

(a) For the same observations, evaluate the two unnormalized weights at $q = 1$. Show $f(1) = 10/(1 + e^{-2/\tau^2})$. What happens as $\tau \to 0$ and as $\tau \to \infty$?

(b) Define scores $s_j = -\|q - k_j\|_2^2/(2\tau^2)$. Using $\mathrm{softmax}(s)_j = e^{s_j}/\sum_l e^{s_l}$, show $\alpha = \mathrm{softmax}(s)$.

(c) Expand $\|q - k_j\|_2^2 = \|q\|_2^2 + \|k_j\|_2^2 - 2q^\top k_j$. Since softmax is unchanged by a common additive constant, prove

$$\alpha_j = \mathrm{softmax}\left(\left(\frac{q^\top k_l}{\tau^2} - \frac{\|k_l\|_2^2}{2\tau^2}\right)_{l=1}^{n}\right)_j.$$

(d) If all key-location norms $\|k_j\|_2$ are equal, simplify this to softmax of scaled dot products. If the norms differ, which term must remain? Explain why Gaussian-distance scoring and dot-product scoring are not identical in general.

Translate back into machine-learning language. The output is still a weighted average. The new ingredient is that the input determines the coefficients through a similarity rule.

SELF-CHECK 4.2

1. Let scalar $q = 0$, $k_1 = 0$, and $k_2 = 2$. Dot-product scores tie. Do Gaussian-distance weights also tie for finite $\tau > 0$? Identify the term that distinguishes the locations.
2. If every location and the query are multiplied by 2, what change to bandwidth τ leaves the Gaussian weights unchanged? Interpret bandwidth as a distance scale.

4.3 Name the three roles: query, key, and value

SETUP: SEPARATE MATCHING FROM WHAT IS AVERAGED

Take $q, k_1, \ldots, k_n \in \mathbb{R}^{d_k}$ and $v_1, \ldots, v_n \in \mathbb{R}^{d_v}$. Assign each pair (q, k_j) a real **attention score** s_j. For now use the dot product $s_j = q^\top k_j$. Define

$$\alpha_j = \frac{e^{s_j}}{\sum_l e^{s_l}}, \qquad o = \sum_j \alpha_j v_j.$$

Now name the roles. The vector q is the **query**; each k_j is a **key** used for comparison with that query; v_j is the corresponding **value**, the vector actually averaged. The normalized coefficients α_j are **attention weights**. The map producing o is **attention pooling**.

The query and keys need equal dimensions for this dot product; values can have a different dimension. "Query" does not mean a literal question in natural language, and "value" is not a probability or the value of a loss.

Worked scalar example. If two scores are $s_1 = \log 3$, $s_2 = 0$, the attention weights are $(3/4, 1/4)$. With scalar values $v_1 = 2$, $v_2 = 6$, the output is 3. Changing the scores changes how much of each value is used.

Guided questions

(a) Let $q = (\log 3, 0)^\top$, $k_1 = (1, 0)^\top$, $k_2 = (0, 1)^\top$, and $v_1 = (2, 0)^\top$, $v_2 = (0, 4)^\top$. Compute the scores, attention weights, and output vector.

(b) Keep q, k_1, k_2 fixed and exchange the two values. Which intermediate quantities stay the same and which output changes? Now exchange the two *key–value pairs* together. Does that merely reorder a sum?

(c) If all keys are identical, what are the attention weights? If all values are identical, can the output depend on the query, regardless of the keys?

(d) The **convex hull** of the values is the set of all their convex combinations. Show o belongs to this set. Must o be one of the values? Must its coordinates be nonnegative?

TWO DIFFERENT USES OF THE WORD "WEIGHT"

An attention weight α_j is a number calculated from the current query and keys. It is not usually an independently stored trainable parameter. The trainable weight matrices in the next setup determine how queries, keys, and values are constructed.

Translate back into machine-learning language. Explain attention once as a normalized weighted sum and once using query/key/value terminology. Name which objects determine the coefficients and which objects are combined.

4.3 / MAKE THE REPRESENTATIONS LEARNABLE

SETUP: LEARNED LINEAR PROJECTIONS

Let a query-source representation be $x \in \mathbb{R}^{d_{\text{in}}}$ and source representations be $x_1, \ldots, x_n \in \mathbb{R}^{d_{\text{in}}}$. Choose trainable matrices

$$W_Q, W_K \in \mathbb{R}^{d_k \times d_{\text{in}}}, \qquad W_V \in \mathbb{R}^{d_v \times d_{\text{in}}},$$

and compute $q = W_Q x$, $k_j = W_K x_j$, $v_j = W_V x_j$. These maps are called **query, key, and value projections**. "Projection" here is standard ML terminology for these learned maps; the matrices need not satisfy $W^2 = W$, be square, or be orthogonal. Biases are omitted in this sheet's attention maps.

The matrices are shared across positions and fixed during a forward pass. The vectors they produce change with the input. In **self-attention**, queries, keys, and values are all constructed from the same input sequence, though generally with different matrices. In **cross-attention**, the query-source sequence differs from the key/value-source sequence. We use equal source feature widths here for simplicity, not as a general requirement.

Worked dimensions. If $d_{\text{in}} = 3$, $d_k = 2$, and $d_v = 4$, then W_Q and W_K are 2×3 matrices and W_V is 4×3. Scores are scalars and the weighted output is in $\mathbb{R}^4$.

Guided questions: connect the operators to parameters

(e) Check each multiplication's dimensions and count all trainable scalar entries in these three matrices. Does that count depend on the number n of source positions?

(f) Substitute the definitions to show

$$q^\top k_j = x^\top W_Q^\top W_K x_j.$$

Which two matrices control this matching rule? With their values fixed, is the score separately linear in x and x_j?

(g) Take $d_{\text{in}} = 2$, $d_k = d_v = 1$, with

$$W_Q = \begin{pmatrix} 1 & 0 \end{pmatrix}, \quad W_K = \begin{pmatrix} 0 & 1 \end{pmatrix}, \quad W_V = \begin{pmatrix} 1 & 1 \end{pmatrix}.$$

Use query-source $x = (1, 0)^\top$ and source vectors $x_1 = (2, 0)^\top$, $x_2 = (0, 1)^\top$. Compute q, k_1, k_2, v_1, v_2, then attention weights and output. Here no square-root scaling is needed because $d_k = 1$.

(h) Hold W_Q, W_K fixed and change W_V. Which quantities can change? Conversely, hold W_V fixed and change W_Q. Explain why a value projection is not a vocabulary output head.

Translate back into machine-learning language. The learned parameters define how to represent and compare inputs. They do not store a separate attention distribution for every possible context.

SELF-CHECK 4.3

1. With a query and keys fixed, replace every value v_j by $v_j + c$. What happens to the attention weights and output? Do not assume c is nonnegative.
2. A sequence doubles in length but keeps the same projection dimensions. Which changes: the number of stored projection parameters, the number of keys and values, or the number of attention coefficients for one query?

4.4 Why divide a dot product by $\sqrt{d_k}$?

PROBABILITY REVIEW AND AN EXPLICIT TOY ASSUMPTION

For a scalar random variable Z, its expectation $\mathbb{E}[Z]$ is its probability-weighted average, and $\mathrm{Var}(Z) = \mathbb{E}[(Z - \mathbb{E}[Z])^2] = \mathbb{E}[Z^2] - (\mathbb{E}[Z])^2$. Independence implies $\mathbb{E}[UV] = \mathbb{E}[U]\mathbb{E}[V]$ when the moments exist. Variances of independent summands add, and $\mathrm{Var}(cZ) = c^2\,\mathrm{Var}(Z)$.

For this calculation only, assume the $2d_k$ scalar coordinates $q_1, \ldots, q_{d_k}, k_1, \ldots, k_{d_k}$ are *mutually independent*, each with mean zero and variance one. Set $S = q^\top k = \sum_{r=1}^{d_k} q_r k_r$. These are modeling assumptions for a scale calculation, not identities satisfied by every learned network.

Worked one-coordinate check. If q_1 and k_1 are independent and each equals 1 or -1 with probability $1/2$, their product also has mean zero and variance one.

Guided questions

(a) Show $\mathbb{E}[q_r k_r] = 0$ and $\mathbb{E}[(q_r k_r)^2] = 1$. Conclude $\mathrm{Var}(q_r k_r) = 1$. Identify every use of independence.

(b) Show $\mathbb{E}[S] = 0$ and $\mathrm{Var}(S) = d_k$. Hence compute $\mathrm{Var}(S/\sqrt{d_k})$. This motivates **scaled dot-product attention**:

$$s_j = \frac{q^\top k_j}{\sqrt{d_k}}, \qquad \alpha = \mathrm{softmax}(s), \qquad o = \sum_j \alpha_j v_j.$$

(c) For two scores $(t, 0)$, the first softmax weight is $a(t) = e^t/(1 + e^t)$. Compute $a'(t)$ and its limit as $t \to +\infty$. This is an example of **saturation**: a normalized weight can become insensitive to changes in the score.

(d) Break the assumption deliberately. Take $k = q$, with independent coordinates of q equal to ± 1. Compute $q^\top k$. Which cross-vector independence assumption fails? Why does the previous zero-mean calculation no longer apply?

WHAT THE SCALING ARGUMENT DOES NOT CLAIM

The square-root factor controls variance in the stated independent-coordinate model. Real queries and keys can be correlated, especially when produced from the same input. The argument does not guarantee unit-variance scores after training, prevent all saturation, or prove successful optimization.

A small softmax Jacobian also does not imply every composite loss derivative is small: the softmax–cross-entropy combination in Meeting 3 has derivative $p - \mathbf{y}$.

Translate back into machine-learning language. Explain the scaling as a variance normalization under assumptions, not a theorem that attention scores always have a prescribed distribution.

SELF-CHECK 4.4

1. Under the stated independent unit-variance assumptions with $d_k = 64$, what are the variances of S, $S/8$, and $S/64$? Which has unit variance?
2. If $q = k$ is the same random vector, is the independent-query/key calculation automatically valid? Give the concrete ± 1 example rather than merely saying that correlation matters.

4.5 Train one attention parameter

SETUP: USE THE SAME CHAIN RULE AS IN MEETING 2

Consider one scalar query-source input $x = 1$ and two scalar sources $x_1 = 1$, $x_2 = 0$. The only trainable parameter is the scalar query-projection weight w. Hold the key and value projections fixed at 1:

$$q = wx = w, \qquad k_1 = v_1 = 1, \qquad k_2 = v_2 = 0.$$

Here $d_k = d_v = 1$, so the square-root scale factor is 1. The scores are $s_1 = w$, $s_2 = 0$; the weights and output are

$$\alpha_1 = \frac{e^w}{e^w + 1}, \qquad \alpha_2 = 1 - \alpha_1, \qquad o = \alpha_1 v_1 + \alpha_2 v_2 = \alpha_1.$$

Use fixed target $y = 1$ and loss $\ell(w) = \frac{1}{2}(o - y)^2$. The attention weights are recomputed from w; they are not additional independent parameters.

Worked forward start. At $w = 0$, the two scores agree, so $\alpha_1 = \alpha_2 = 1/2$, $o = 1/2$, and $\ell = 1/8$. We now ask how a prediction error changes the projection weight, and hence the next forward pass.

Guided questions

(a) Differentiate $o = e^w/(1 + e^w)$ using the quotient rule. Express do/dw in terms of o alone.

(b) Apply the chain rule to $\ell = \frac{1}{2}(o - 1)^2$ and derive

$$\frac{d\ell}{dw} = (o - 1)o(1 - o).$$

Evaluate this derivative at $w = 0$. Separate the local derivative do/dw from the arriving loss derivative $\partial\ell/\partial o$.

(c) Take a gradient-descent step $w^+ = w - \eta\, d\ell/dw$ with $\eta = 1$. Recompute the scores, weights, output, and loss using w^+. A calculator is optional; you may leave exponentials unevaluated. Did this step increase or decrease the weight on the value matching the target?

(d) State which quantities were stored as parameters, which were held fixed as data or settings, and which were recomputed. Why is the update an example of learning an attention rule, rather than directly storing a new α_1?

Translate back into machine-learning language. Connect the whole chain: a fixed target defines a loss; backpropagation supplies a derivative of a projection weight; the optimizer changes that weight; the next forward pass produces a different attention distribution. This example does not promise improvement for every learning rate or every attention model.

SELF-CHECK 4.5

1. Change the target to $y = 0$, leaving $w = 0$ and all sources unchanged. What is $d\ell/dw$ now, and in which direction does gradient descent change the first attention weight?
2. Instead set both values to 1 and target $y = 0$, while keeping the keys as before. Can changing w reduce the loss? Explain how a nonzero prediction error can coexist with zero score derivatives.

Optional extension: a backward pass through attention

4.E / DIFFERENTIATE THE WEIGHTED SUM

Keep values $v_j \in \mathbb{R}^{d_v}$ and scores $s_j \in \mathbb{R}$ as separate inputs to

$$\alpha = \operatorname{softmax}(s), \qquad o = \sum_j \alpha_j v_j.$$

Suppose a later scalar loss supplies $g = \nabla_o \ell \in \mathbb{R}^{d_v}$. For now, varying a score does not also vary a value. We will combine these partial derivatives with the learned projections only after deriving them.
The softmax derivative is $\partial \alpha_j / \partial s_l = \alpha_j(\delta_{jl} - \alpha_l)$, where $\delta_{jl} = 1$ for equal indices and 0 otherwise. For a vector output, $\partial o / \partial s_l$ is the vector of its coordinate derivatives.

Worked interpretation. If all values equal a common vector v, the output always equals v for any scores. Therefore changing the scores alone cannot change this output.

Guided questions

(a) Differentiate with respect to v_j, holding scores fixed, to get $\nabla_{v_j} \ell = \alpha_j g$.

(b) Differentiate the output with respect to s_l. Separate the term containing δ_{jl} and prove

$$\frac{\partial o}{\partial s_l} = \alpha_l (v_l - o), \qquad \frac{\partial \ell}{\partial s_l} = \alpha_l \, g^\top (v_l - o).$$

(c) Revisit the identical-values case. Does the formula recover the zero derivative? Show also $\sum_l \partial \ell / \partial s_l = 0$ and interpret it using softmax's common-shift invariance.

(d) Use scalar values $(2, 6)$, weights $(3/4, 1/4)$, and loss $\ell = \frac{1}{2}(o - 4)^2$. Find o, g, and both score derivatives. In a small direct score update, which value receives more relative attention?

(e) Finally use $s_j = q^\top k_j / \sqrt{d_k}$ and set $r_j = \partial \ell / \partial s_j$. Derive

$$\nabla_q \ell = \frac{1}{\sqrt{d_k}} \sum_j r_j k_j, \qquad \nabla_{k_j} \ell = \frac{r_j}{\sqrt{d_k}} q.$$

What local affine-layer rule would turn these into gradients of W_Q and W_K?

Translate back into machine-learning language. Attention is trained by the same chain rule as an affine/ReLU network. A source vector can participate as both a key and a value; derivatives from its several uses must then be added.

SELF-CHECK 4.E

1. If a single source vector participates as both a key and a value, is its derivative given by just the value path? State how the separate path contributions combine.

Discussion checkpoint and selected hints

READY TO MOVE ON WHEN...

Construct an attention output from scores and values, distinguish projection parameters from computed weights, and use 4.5 to explain how a loss changes one query-projection parameter and the next attention distribution.
State the assumptions of the square-root scaling argument; the full matrix backward pass is optional.
You may consult definitions and notation. Explain the choices without following a worked solution line by line.
If a self-check exposes a gap, return to that problem's setup; optional extensions do not block progress.

CHECK YOUR TRANSLATIONS

Scores are real compatibility numbers. Attention weights normalize those scores over source positions. Values are the vectors combined, and the output is a feature vector, not generally a probability vector over vocabulary items.
Projection matrices are parameters. Queries, keys, values, and attention weights are computed from the current inputs. A "projection" in this usage need not be idempotent.
Gaussian-distance and dot-product scores agree up to a softmax-irrelevant constant only under appropriate conditions on key norms. The square-root scaling is motivated by a stated variance model.

Suggested discussion. Explain the progression from a fixed average to an input-dependent average to learned projections. Give a concrete example in which changing a value changes the output without changing the attention weights.

4.1(d). Each coordinate solves the same scalar quadratic with the same coefficients a_j.

4.2(c)–(d). The term $-\|q\|_2^2/(2\tau^2)$ is common to every score. The key-norm term is common only when the key norms agree.

4.3(b). Changing which value belongs to a key changes the association. Reordering complete key–value pairs merely reindexes the sum.

4.3(g). The two keys are 0 and 1, the values are 2 and 1, and the query is 1.

4.4(b). Products built from disjoint independent coordinate pairs are independent. Their variances, not their standard deviations, add.

4.E(b). Use $\sum_j \alpha_j v_j = o$ after applying the softmax derivative.

Optional reading: reconnect with standard sources

[1] Zhang, Lipton, Li, and Smola, *Dive into Deep Learning*. §11.1, Queries, Keys, and Values; §11.2, Attention Pooling by Similarity, especially §§11.2.1–11.2.2; §11.3, Attention Scoring Functions, especially §§11.3.1 and 11.3.3.

[2] Vaswani et al., *Attention Is All You Need* (2017). §3.2 and §3.2.1, including the scaling footnote. Only the attention definition and variance motivation are needed, not the whole architecture.

[3] Prince, *Understanding Deep Learning* (2023). §§12.2–12.3: dot-product self-attention and its extensions. For probability review, Deisenroth, Faisal, and Ong, *Mathematics for Machine Learning*, §6.4, gives summary statistics and independence.

Readings supply context, not missing definitions. Exercises and numerical examples are original to this DRP. Prince locators refer to the 2023 first-edition organization; D2L locators refer to its 1.0.3 online edition. Use section titles when editions differ.